

OPTIMIZED HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: Romano A. Gabrillo

Program: Doctor of Engineering with specialization in Computer Engineering

Use this form to record and justify the hyperparameter values actually used in the CNN experiment. For the tuned model, the “Chosen Value” records the best configuration returned by Keras Tuner (Hyperband), while the notebook defaults record the original manually specified CNN configuration. All values and results below are taken from the file “CNN Activity with Hyperparameters Tuning.ipynb”.

1. Project / Experiment Information

Project / Experiment Title	CNN Activity with Hyperparameters Tuning
Model Architecture	2D Convolutional Neural Network (CNN) with Keras Tuner Hyperband optimization
Dataset Used	Kaggle Dogs vs. Cats image dataset (Dog/ and Cat/ folders)
Notebook / File Reference	CNN Activity with Hyperparameters Tuning.ipynb
Prepared By	Romano Gabrillo
Date	October 3, 2026
Experiment / Trial No.	Keras Tuner Hyperband — Best Configuration / Final Tuned Model

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	50 / 80 / 100 / 128 / 150	100	100 × 100	The notebook explicitly sets IMG_SIZE = 100 and resizes every readable image to 100×100. The tuned model does not change IMG_SIZE.
Color Mode	Grayscale / RGB	Grayscale	Grayscale (1 channel)	OpenCV reads images with cv2.IMREAD_GRAYSCALE and X is reshaped to (-1, 100, 100, 1). The tuner uses the same X; no RGB conversion is implemented.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	The CNN section divides X by 255.0 so pixel intensities are scaled from 0–255 to 0–1.
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes	The complete training_data list is shuffled with random.shuffle(training_data) before X and y are created.
Held-out Test Set (separate from training)	Yes / No	No	No	The notebook does not create an independent test set. It creates an 80/20 train-validation split for tuning and final validation. Therefore, the reported final 0.8488 is validation performance, not independent test-set performance.

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Validation Split / Validation Set	0.10 / 0.20 / 0.30	0.20	0.20 (5,000 validation; 19,997 training)	Keras Tuner uses train_test_split with test_size=0.20, random_state=42, and stratify=y. The resulting split is 19,997 training and 5,000 validation images.

3. Model Architecture Hyperparameters

Trial 1

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Block 1 – Filters	32–256 (step 32)	2	96	Best tuner value. Provides substantially more first-layer feature capacity than the original 2-filter baseline.	—	—
Conv2D Block 1 – Kernel Size	3×3 / 5×5	3×3	3×3	Best tuner value; the first block uses a smaller kernel for local edge/texture extraction.	—	—
Conv2D Block 1 – Activation	ReLU / tanh / ELU / leaky ReLU	ReLU	ReLU	The tuner model uses ReLU after every convolutional layer.	—	—
Pooling Block 1	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	The architecture keeps 2×2 max pooling after each convolutional block to reduce spatial dimensions.	—	—
Conv2D Block 2 – Filters	32–256 (step 32)	64	96	Best tuner value; increases representational capacity while remaining within the defined search space.	—	—
Conv2D Block 2 – Kernel Size	3×3 / 5×5	3×3	5×5	Best tuner value selected by validation accuracy during Hyperband search.	—	—
Conv2D Block 3 – Filters	32–256 (step 32)	64	32	Best tuner value. The search selected fewer filters in the third block, reducing later-stage model complexity.	—	—
Conv2D Block 3 – Kernel Size	3×3 / 5×5	3×3	5×5	Best tuner value selected during Hyperband search.	—	—
Dense Layer 1 – Units	32–256 (step 32)	64	256	The tuner selected the maximum width in the defined dense-layer search range for the first dense layer.	—	—
Dense Layer 2 – Units	32–256 (step 32)	64	128	A second dense layer was selected by the tuner, providing additional nonlinear representation before classification.	—	—
Dropout Rate	0.0–0.5 (step 0.1)	0.0 / none	0.2	The tuned model adds 20% dropout to reduce reliance on individual dense-layer activations and help control overfitting.	—	—
Output Layer Activation	Sigmoid / Softmax	Sigmoid	Sigmoid	The task has two classes (Dog and Cat), so a single sigmoid output is used for binary classification.	—	—

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	Appropriate for a binary target with one sigmoid output.	—	0.8488*
Optimizer	Adam / RMSprop	adam	Adam	The tuner searched Adam and RMSprop and selected Adam for the best validation accuracy.	—	0.8488*
Learning Rate	1e-4–1e-2, log scale	0.001 (baseline)	0.000596921247059964	Automatically selected by Hyperband from the log-scaled search interval; approximately 5.9692×10^{-4} .	—	0.8488*
Batch Size	32 / 64 / 128	128 (baseline)	64	Automatically selected by the tuner to balance gradient estimation, memory use, and training speed.	—	0.8488*
Tuner Search Epochs	15 maximum	5 (baseline)	15 maximum	Hyperband was configured with max_epochs=15. EarlyStopping with patience=3 was used during the search.	—	0.8488*
Final Training Epochs	Up to 50	5 (baseline)	Up to 50	The winning model was retrained for up to 50 epochs with EarlyStopping (patience=5) and ReduceLROnPlateau.	0.9022†	0.8488
Early Stopping	Yes/No	No	Yes; patience=5, restore_best_weights=True	Final training stops when validation loss fails to improve for the specified patience and restores the best validation-loss weights.	0.9022†	0.8488
ReduceLROnPlateau	Yes/No	No	Yes; factor=0.5, patience=2, min_lr=1e-6	The learning rate is reduced by half when validation loss plateaus, with a minimum learning rate of 1e-6.	0.9022†	0.8488
Tuning Algorithm	Manual / automated	Manual baseline	Keras Tuner Hyperband	The notebook searches architecture and training hyperparameters automatically using validation accuracy as the objective.	—	Best search val. accuracy = 0.8684

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy (reported at epoch 6 / restored-best-val-loss epoch)	0.9022†	Final model was restored to the best validation-loss weights from epoch 6; the notebook does not separately call evaluate() on X_train after restoration.
Validation Accuracy	0.8488 (84.88%)	5,000-image validation set; this is not an independent test set.
Training Loss (epoch 6)	0.2340†	Reported during epoch 6; final post-restoration training loss was not separately evaluated.
Validation Loss (best)	0.3440†	Lowest validation loss shown in the final training log (epoch 6); EarlyStopping restores these weights.
Validation AUC	0.9335	Computed by the notebook's ROC/AUC evaluation and printed in the baseline-vs-tuned comparison.
Confusion Matrix	[[2294, 206], [550, 1950]]	Rows = true labels [Dog, Cat]; columns = predicted labels [Dog, Cat].
Macro F1-score (derived)	0.8481	Derived from the reported confusion matrix; not explicitly printed by the notebook.
Dog precision / recall / F1 (derived)	0.8066 / 0.9176 / 0.8585	Derived from the reported confusion matrix.
Cat precision / recall / F1 (derived)	0.9045 / 0.7800 / 0.8376	Derived from the reported confusion matrix.

5. a. Paste here all screenshots/figures of the tuned model

Loading Dataset and CNN

Rolando A. Pula

This notebook is a runnable version of the "Loading Dataset and CNN" slide deck. It walks through:

1. Loading a Dogs vs. Cats image dataset with OpenCV
2. Preprocessing the images (grayscale, resize, shuffle)
3. Saving/loading the prepared data with `pickle`
4. Building and training a Convolutional Neural Network (CNN) with TensorFlow/Keras to classify dogs vs. cats

Before you run this notebook: download the Kaggle "Dogs vs. Cats" dataset (e.g. <https://www.kaggle.com/c/dogs-vs-cats/data>, or the mirrored `salader/dogs-vs-cats` dataset on Kaggle) and unzip it so you have two folders, `Dog/` and `Cat/`, each full of `.jpg` images. Then set `DATADIR` below to point at the parent folder that contains `Dog/` and `Cat/`.

```
[1]: # Run this once if the packages below aren't already installed.
# $pip install opencv-python numpy matplotlib tensorflow
```

Requirements for Image Processing

- **GPU** (optional, speeds up training but not required)
- **OpenCV** (`cv2`) – for image processing
- **os module** – lets Python interface with the OS running on your computer (already in the standard library)
- **Data** – we'll use the Kaggle "Dogs vs. Cats" dataset

Run the cell below once to install the packages this notebook needs (skip it if they're already installed in your environment).

- **Data** – we'll use the Kaggle "Dogs vs. Cats" dataset

Run the cell below once to install the packages this notebook needs (skip it if they're already installed in your environment).

Checking and installing OpenCV

The original slides show installing OpenCV through **Anaconda Navigator** (Environments tab → search `opencv` → Install). If you're not using Anaconda, the `pip install opencv-python` command in the cell above does the same thing.

Calling the Necessary Modules

```
[2]: import numpy as np
import matplotlib.pyplot as plt
import os
import cv2
```

Data Preparation

Set `DATADIR` to the folder on your machine that contains the `Dog/` and `Cat/` subfolders (this replaces the original slide's hard-coded Windows path, `C:\Users\admin\Desktop\AMEN\Deep_Learning_class\Kaggle_dataset\PetImages`, with something you can edit for your own machine).

```
[3]: # from google.colab import drive
# drive.mount('/content/drive')

import os

print(os.getcwd())
data_path = r"C:\Users\hp\Documents\Study\2026-2027 1st Sem\DOES IC3 -Bioinformatics"
```

```
print(os.getcwd())
data_path = r"C:\Users\hp\Documents\Study\2026-2027 1st Sem\DOES 1C3 -Bioinformatics"
Mounted at /content/drive
```

Unzipping the Dataset

Given that the dataset is in a .zip file in Google Drive, we need to unzip it to a local directory before processing. I'll unzip PetImages.zip to /content/ and then set DATADIR to this new location.

```
[4]: import zipfile
import os

# Path to the zip file in Google Drive

zip_file_path = r"C:\Users\hp\Documents\Study\2026-2027 1st Sem\DOES 1C3 -Bioinformatics\PetImages.zip'

# Destination directory to unzip the files
destination_dir = r"C:\Users\hp\Documents\Study\2026-2027 1st Sem\DOES 1C3 -Bioinformatics\PetImages'

# Create the destination directory if it doesn't exist
os.makedirs(destination_dir, exist_ok=True)

# Unzip the file
with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
    zip_ref.extractall(destination_dir)

print(f"Unzipped '{zip_file_path}' to '{destination_dir}'")

# The dataset will be unzipped into a folder named 'PetImages' inside destination_dir
# Update DATADIR to point to this unzipped location
```

```
# The dataset will be unzipped into a folder named 'PetImages' inside destination_dir
# Update DATADIR to point to this unzipped location
```

```
Unzipped '/content/drive/MyDrive/PetImages.zip' to '/content/'
```

```
[5]: DATADIR = r"C:\Users\hp\Documents\Study\2026-2027 1st Sem\DOES 1C3 -Bioinformatics\PetImages" # <- change this to the path that contains your Dog/ and
# this is the directory of your datasets
CATEGORIES = ["Dog", "Cat"] # Defining the label as "Dog" or "Cat"

for category in CATEGORIES:
    path = os.path.join(DATADIR, category) # path to cats or dogs dir
    for img in os.listdir(path):
        img_array = cv2.imread(os.path.join(path, img), cv2.IMREAD_GRAYSCALE)
        if img_array is None:
            # a handful of files in the Kaggle dataset are corrupted/unreadable - skip them
            continue

        # reading the image in the path and transforming it into an array
        # cv2.IMREAD_GRAYSCALE converts the image to grayscale so the CNN only has
        # to learn from one color channel instead of three (faster, smaller model)

        plt.imshow(img_array, cmap="gray")
        plt.show()
        break
```



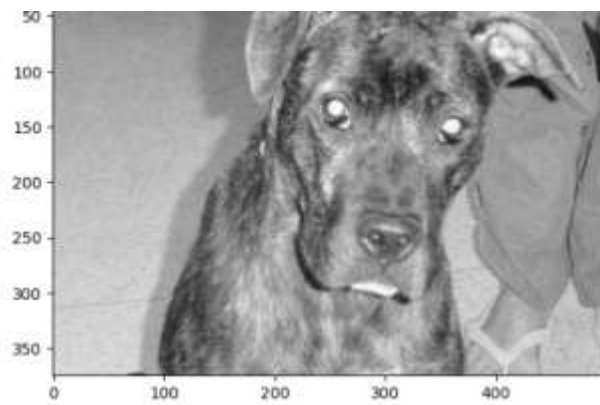


Image Inspection

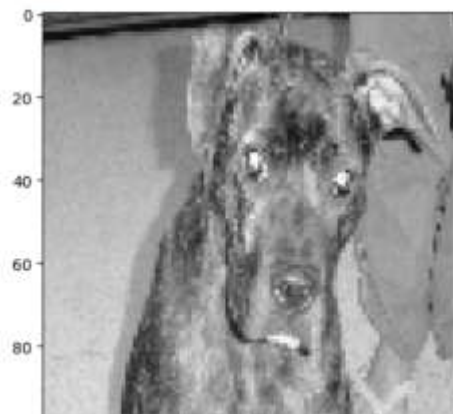
```
[6]: print(img_array)      # Displaying the image in array form (just for checking)
     print(img_array.shape) # checking the resolution of the image

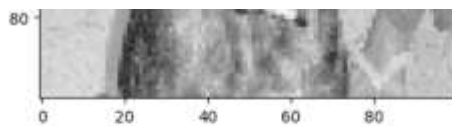
[[131 135 138 ... 136 123 115]
 [129 132 134 ... 138 125 116]
 [129 132 134 ... 148 127 117]
 ...
 [225 225 225 ... 217 218 219]
 [224 224 224 ... 215 216 217]
 [223 223 224 ... 209 210 211]]
(375, 500)
[225 225 225 ... 217 218 219]
[224 224 224 ... 215 216 217]
[223 223 224 ... 209 210 211]]
(375, 500)
```

Image Resizing

```
[7]: IMG_SIZE = 100 # initializing variable "IMG_SIZE" as 100

new_array = cv2.resize(img_array, (IMG_SIZE, IMG_SIZE))
plt.imshow(new_array, cmap='gray')
plt.show()
```





Assigning Training Data

```
[8]: training_data = [] # assigning training data

def create_training_data():
    for category in CATEGORIES:
        path = os.path.join(DATA_DIR, category) # path to cats or dogs dir
        class_num = CATEGORIES.index(category)
        for img in os.listdir(path):
            try:
                img_array = cv2.imread(os.path.join(path, img), cv2.IMREAD_GRAYSCALE)
                new_array = cv2.resize(img_array, (IMG_SIZE, IMG_SIZE))
                training_data.append([new_array, class_num])
            except Exception as e:
                pass

create_training_data()
```

Checking the Number of Training Data

```
[9]: print(len(training_data))

24997
```

```
[9]: print(len(training_data))

24997
```

Randomizing/Shuffling the Data

```
[10]: import random
      random.shuffle(training_data)

      for sample in training_data[:10]:
          print(sample[1])

0
1
0
0
1
1
1
0
0
1
```

Assigning Variables for Features and Labels

```
[11]: X = [] # feature set
      y = [] # labels

      for features, label in training_data:
          X.append(features)
          y.append(label)

      X = np.array(X).reshape(-1, IMG_SIZE, IMG_SIZE, 1) # 1 is for grayscale
```

```

X.append(features)
y.append(label)

X = np.array(X).reshape(-1, IMG_SIZE, IMG_SIZE, 1) # 1 is for grayscale
y = np.array(y) # Keras expects the labels as an array too

```

Using the Pickle Module

```

[11]: import pickle # pickle lets you save your training data, avoiding recreating it
      # from scratch every time you want to adjust the model

      with open('X.pickle', 'wb') as f:
          pickle.dump(X, f)

      with open('y.pickle', 'wb') as g:
          pickle.dump(y, g)

```

Using Pickle (Loading It Back)

```

[12]: pickle_in = open('X.pickle', 'rb')
      X = pickle.load(pickle_in)

      X[1]

[13]: array([[[ 29],
               [ 31],
               [ 33],
               ...,
               [141],
               [109],
               [ 85]],

              [[ 47],
               [ 46],
               [ 46],
               ...,
               [106],
               [131],
               [ 87]],

              [[ 46],
               [ 48],
               [ 50],
               ...,
               [157],
               [145],
               [ 89]],

              ...,

              [[ 90],
               [ 94],
               [ 92],
               ...,
               [ 37],
               [ 48],
               [ 58]],

              [[ 91],
               [ 96],
               [ 94],
               ...,
               [ 32],
               [ 44],
               [ 55]],

```

```

...
[ 31],
[ 44],
[ 55]],

[[ 95],
[ 90],
[100],
...
[ 35],
[ 40],
[ 50]]], dtype=uint8)

```

What is a CNN?

CNN stands for **Convolutional Neural Network** — a type of neural network especially well suited to image data because it learns spatial patterns (edges, textures, shapes) through convolution and pooling layers instead of treating every pixel independently.

Further reading:

- <https://medium.com/data-science-group-ilr/building-a-convolutional-neural-network-in-python-with-tensorflow-d251c3ca8117>
- <https://hackernoon.com/what-is-a-capsnet-or-capsule-network-2bfb648769cc>

Using a CNN for Classification of Images

```

[14]: import tensorflow as tf
      from tensorflow.keras.models import Sequential
      from tensorflow.keras.layers import Input, Dense, Dropout, Activation, Flatten, Conv2D, MaxPooling2D
      import pickle

      X = pickle.load(open('X.pickle', 'rb'))
      y = pickle.load(open('y.pickle', 'rb'))

      from tensorflow.keras.layers import Input, Dense, Dropout, Activation, Flatten, Conv2D, MaxPooling2D
      import pickle

      X = pickle.load(open('X.pickle', 'rb'))
      y = pickle.load(open('y.pickle', 'rb'))

```

Building, Compiling, and Training the CNN

`X = X/255.0` normalizes the pixel values (0-255) down to the 0-1 range, which helps the network train faster and more stably.

Training for the full 200 epochs on the whole dataset can take a long time on CPU. Reduce `epochs` (e.g. to 10) for a quick test run, and increase it back up once you know the pipeline works.

```

[14]:
[15]: X = X / 255.0

      model = Sequential()
      model.add(Input(shape=X.shape[1:]))

      model.add(Conv2D(2, (3, 3)))
      #model.add(Conv2D(64, (3, 3)))
      model.add(Activation("relu"))
      model.add(MaxPooling2D(pool_size=(2, 2)))

      model.add(Conv2D(64, (3, 3)))
      model.add(Activation("relu"))
      model.add(MaxPooling2D(pool_size=(2, 2)))

      model.add(Conv2D(64, (3, 3)))
      model.add(Activation("relu"))
      model.add(MaxPooling2D(pool_size=(2, 2)))

```

```

model.add(Conv2D(64, (3, 3)))
model.add(Activation("relu"))
model.add(MaxPooling2D(pool_size=(2, 2)))

model.add(Flatten())
model.add(Dense(64))
model.add(Dense(64))
model.add(Dense(1))
model.add(Activation('sigmoid'))

model.compile(loss="binary_crossentropy",
              optimizer="adam",
              metrics=['accuracy'])

```

```
[16]: model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 98, 98, 2)	20
activation (Activation)	(None, 98, 98, 2)	0
max_pooling2d (MaxPooling2D)	(None, 49, 49, 2)	0
conv2d_1 (Conv2D)	(None, 47, 47, 64)	1,216
activation_1 (Activation)	(None, 47, 47, 64)	0
conv2d_1 (Conv2D)	(None, 47, 47, 64)	1,216
activation_1 (Activation)	(None, 47, 47, 64)	0
max_pooling2d_1 (MaxPooling2D)	(None, 23, 23, 64)	0
conv2d_2 (Conv2D)	(None, 21, 21, 64)	36,928
activation_2 (Activation)	(None, 21, 21, 64)	0
max_pooling2d_2 (MaxPooling2D)	(None, 10, 10, 64)	0
flatten (Flatten)	(None, 6400)	0
dense (Dense)	(None, 64)	409,664
dense_1 (Dense)	(None, 64)	4,160
dense_2 (Dense)	(None, 1)	65
activation_3 (Activation)	(None, 1)	0

Total params: 452,053 (1.72 MB)

Trainable params: 452,053 (1.72 MB)

Non-trainable params: 0 (0.00 B)

```
[17]: history = model.fit(X, y, batch_size=128, epochs=5, validation_split=0.2)
```

```

Epoch 1/5
157/157 — 12s 43ms/step - accuracy: 0.6344 - loss: 0.6281 - val accuracy: 0.7462 - val loss: 0.5314

```

```
[17]: history = model.fit(X, y, batch_size=128, epochs=5, validation_split=0.2)

Epoch 1/5
157/157 — 12s 43ms/step - accuracy: 0.6344 - loss: 0.6281 - val_accuracy: 0.7462 - val_loss: 0.5314
Epoch 2/5
157/157 — 13s 19ms/step - accuracy: 0.7595 - loss: 0.4987 - val_accuracy: 0.7662 - val_loss: 0.4836
Epoch 3/5
157/157 — 3s 22ms/step - accuracy: 0.7933 - loss: 0.4438 - val_accuracy: 0.7758 - val_loss: 0.4835
Epoch 4/5
157/157 — 3s 18ms/step - accuracy: 0.8215 - loss: 0.4009 - val_accuracy: 0.7952 - val_loss: 0.4567
Epoch 5/5
157/157 — 3s 17ms/step - accuracy: 0.8338 - loss: 0.3711 - val_accuracy: 0.8084 - val_loss: 0.4366
```

```
[18]: model.summary()

import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, roc_curve, auc
import numpy as np # numpy is already imported as np earlier

# Plot training accuracy and loss
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label='val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0, 1])
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='loss')
plt.plot(history.history['val_loss'], label='val_loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
```

```
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='loss')
plt.plot(history.history['val_loss'], label='val_loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')

plt.show()

# Get predictions for confusion matrix and ROC
y_pred_proba = model.predict(X).ravel() # .ravel() to flatten to 1D array
y_pred = (y_pred_proba > 0.5).astype(int)

# Confusion Matrix
cm = confusion_matrix(y, y_pred)
print("\nConfusion Matrix:")
print(cm)

plt.figure(figsize=(6, 5))
plt.imshow(cm, interpolation='nearest', cmap=plt.cm.Blues)
plt.title('Confusion Matrix')
plt.colorbar()
tick_marks = np.arange(len(np.unique(y)))
CATEGORIES = ["Dog", "Cat"]
plt.xticks(tick_marks, CATEGORIES)
plt.yticks(tick_marks, CATEGORIES)
plt.ylabel('True label')
plt.xlabel('Predicted label')
fmt = 'd'
thresh = cm.max() / 2.
for i, j in np.ndindex(cm.shape):
    plt.text(j, i, format(cm[i, j], fmt),
             ha="center", va="center")
```

```

thresh = cm.max() / 2.
for i, j in np.ndindex(cm.shape):
    plt.text(j, i, format(cm[i, j], 'f'),
             has="center", vax="center",
             colors="white" if cm[i, j] > thresh else "black")
plt.tight_layout()
plt.show()

# ROC Curve and AUC
fpr, tpr, thresholds = roc_curve(y, y_pred_proba)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc="lower right")
plt.show()

```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 98, 98, 2)	20
activation (Activation)	(None, 98, 98, 2)	0
max_pooling2d (MaxPooling2D)	(None, 49, 49, 2)	0
activation (Activation)	(None, 98, 98, 2)	0
max_pooling2d (MaxPooling2D)	(None, 49, 49, 2)	0
conv2d_1 (Conv2D)	(None, 47, 47, 64)	1,216
activation_1 (Activation)	(None, 47, 47, 64)	0
max_pooling2d_1 (MaxPooling2D)	(None, 23, 23, 64)	0
conv2d_2 (Conv2D)	(None, 21, 21, 64)	36,928
activation_2 (Activation)	(None, 21, 21, 64)	0
max_pooling2d_2 (MaxPooling2D)	(None, 10, 10, 64)	0
flatten (Flatten)	(None, 6400)	0
dense (Dense)	(None, 64)	409,664
dense_1 (Dense)	(None, 64)	4,160
dense_2 (Dense)	(None, 1)	65
activation_3 (Activation)	(None, 1)	0

Total params: 1,356,161 (5.17 MB)

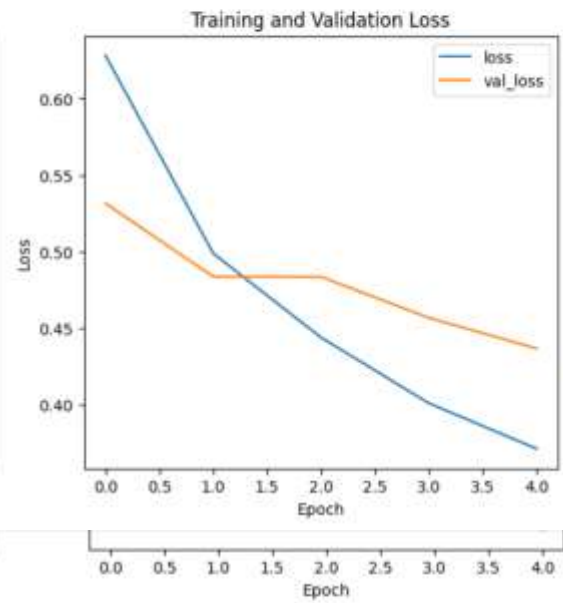
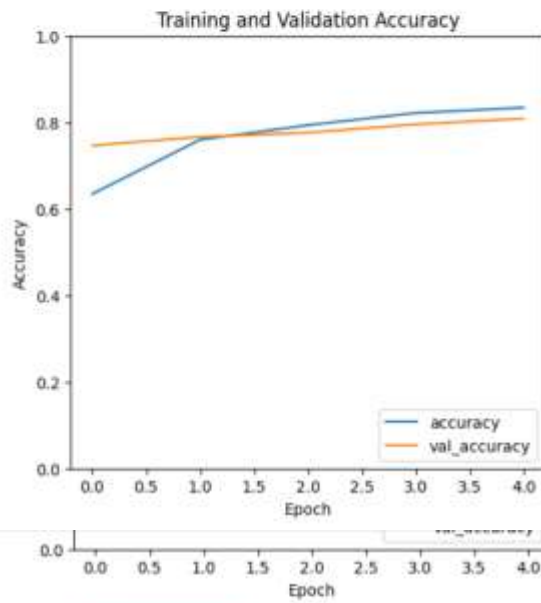
Trainable params: 452,053 (1.72 MB)

Non-trainable params: 0 (0.00 B)

Trainable params: 452,053 (1.72 MB)

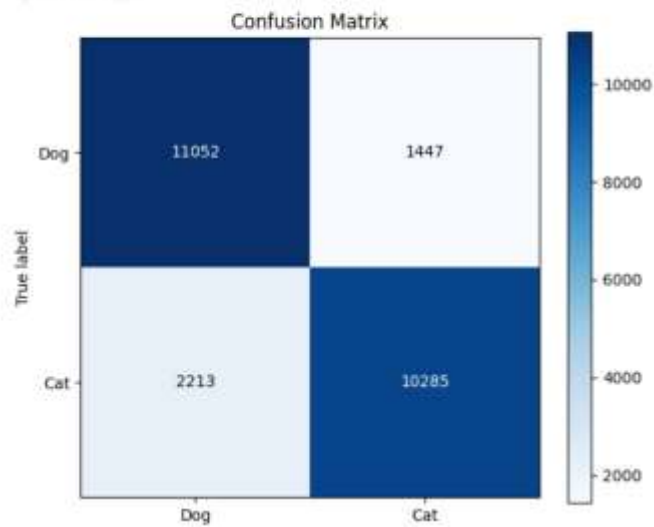
Non-trainable params: 0 (0.00 B)

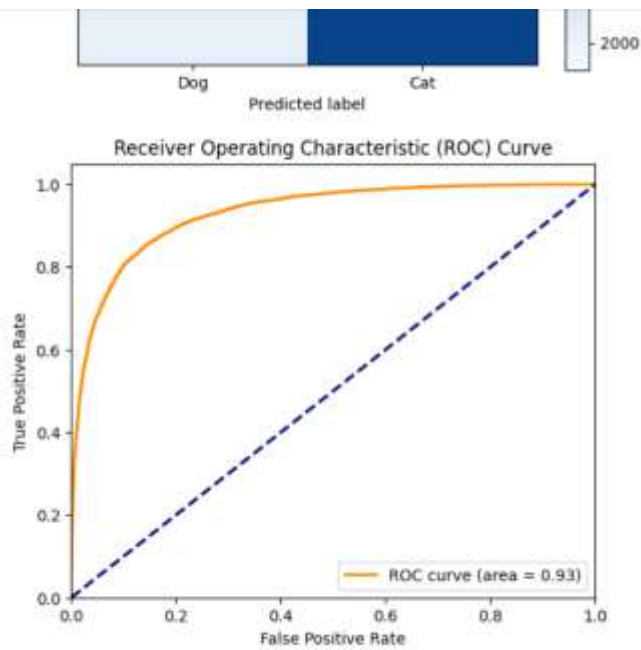
Optimizer params: 984,188 (3.45 MB)



782/782 — 4s 4ms/step

Confusion Matrix:
[[11052 1447]
 [2213 10285]]





```
[19]: import tensorflow as tf
```

False Positive Rate

```
[10]: import tensorflow as tf
```

```
# Define the filename for your model
model_filename = 'dog_cat_classifier.keras' # .keras is the recommended extension for Keras models

# Save the model
model.save(model_filename)
print(f"Model saved to {model_filename}")

# To load the model later (you can do this in a new session or script)
loaded_model = tf.keras.models.load_model(model_filename)
print(f"Model loaded from {model_filename}")

# You can then use the loaded_model for predictions or further evaluation
# For example, let's make predictions with the loaded model and compare
loaded_model_predictions = loaded_model.predict(X)
print("\nExample predictions from loaded model:")
print(loaded_model_predictions[:5])
```

```
Model saved to dog_cat_classifier.keras
Model loaded from dog_cat_classifier.keras
782/782 ————— 3s 3ms/step
```

```
Example predictions from loaded model:
```

```
[[0.00130012]
 [0.9839263 ]
 [0.19602000]
 [0.27930163]
 [0.2685425 ]]
```

Prediction Inference

```
[0.2685425 ]]
```

Prediction Inference

Now that the model is loaded, we can use it to make predictions on new, unseen data. For demonstration purposes, I'll take a few samples from your existing `X` data to illustrate how to get predictions from the `loaded_model`.

```
[20]: # Select a small subset of data for inference (e.g., the first 5 samples)
# In a real scenario, this would be entirely new, unseen test data.
test_samples = X[:5]

# Make predictions using the loaded model
inference_predictions = loaded_model.predict(test_samples)

print("Predictions for the test samples:")
for i, prediction in enumerate(inference_predictions):
    # Assuming binary classification with sigmoid output, values close to 1 are 'Cat', close to 0 are 'Dog'
    predicted_class = 'Cat' if prediction[0] > 0.5 else 'Dog'
    probability = prediction[0] * 100
    print(f"Sample {i+1}: Predicted Class = {predicted_class} (Probability: {probability:.2f}%) ")

# Optionally, display the images and their predictions
plt.figure(figsize=(10, 4))
for i in range(min(5, len(test_samples))):
    plt.subplot(1, 5, i + 1)
    plt.imshow(test_samples[i].reshape(IMG_SIZE, IMG_SIZE), cmap='gray')
    predicted_class = 'Cat' if inference_predictions[i][0] > 0.5 else 'Dog'
    plt.title(f"Pred: {predicted_class}")
    plt.axis('off')
plt.tight_layout()
plt.show()
```

```
1/1 ————— 0s 39ms/step
```

```
predictions: array([0.14, 0.39, 0.19, 0.27, 0.27])
plt.tight_layout()
plt.show()
```

```
1/1 ————— 0s 39ms/step
```

```
Predictions for the test samples:
Sample 1: Predicted Class = Dog (Probability: 0.14%)
Sample 2: Predicted Class = Cat (Probability: 39.39%)
Sample 3: Predicted Class = Dog (Probability: 19.60%)
Sample 4: Predicted Class = Dog (Probability: 27.94%)
Sample 5: Predicted Class = Dog (Probability: 26.85%)
```

Pred: Dog



Pred: Cat



Pred: Dog



Pred: Dog



Pred: Dog



Hyperparameter Optimization with Keras Tuner

The CNN above used one fixed, hand-picked set of hyperparameters (a very thin first conv layer with only 2 filters, no dropout, a single learning rate, and a batch size chosen without any experimentation). This section uses *Keras Tuner* to search over several hyperparameters automatically and picks the configuration that gets the best validation accuracy, instead of guessing.

Hyperparameters tuned in this section:

- Number of convolutional blocks (2–4)
- Filters per convolutional layer (32–256 in steps of 32)

Hyperparameters tuned in this section:

- **Number of convolutional blocks** (2–4)
- **Filters per convolutional layer** (32–256, in steps of 32)
- **Kernel size per convolutional layer** (3×3 or 5×5)
- **Number of dense layers** (1–2) and their **width** (32–256 units)
- **Dropout rate** (0.0–0.5) — the original model had no dropout at all, which is a common cause of overfitting
- **Learning rate** (1e-4 to 1e-2, sampled on a log scale)
- **Optimizer** (Adam vs. RMSprop)
- **Batch size** (32, 64, or 128)

Compute note: hyperparameter search trains many candidate models, so it takes noticeably longer than training a single model. On Colab, switch to a GPU runtime (Runtime → Change runtime type → GPU) before running the cells below. If you just want to see the mechanism work, lower `max_epochs` in the tuner and `epochs` in `tuner.search(...)` first, then raise them once you know it runs end to end.

```
[21]: # Run this once if keras-tuner can't already be installed.
      !pip install -q keras-tuner

      129.4/129.4 kB 5.5 MB/s eta 0:00:00

[22]: import keras_tuner as kt
      from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
      from sklearn.model_selection import train_test_split

      # Keras Tuner re-splits data internally across trials, but we hold out one fixed
      # validation set so every trial (and the final tuned model) is judged on the same
      # unseen data, making the comparison fair.
      X_train, X_val, y_train, y_val = train_test_split(
          X, y, test_size=0.2, random_state=42, stratify=y
      )
      print(f"Train: {X_train.shape[0]} samples | Validation: {X_val.shape[0]} samples")

      X, y, test_size=0.2, random_state=42, stratify=y
      )
      print(f"Train: {X_train.shape[0]} samples | Validation: {X_val.shape[0]} samples")

      Train: 19997 samples | Validation: 5000 samples

[23]: def build_model(hp):
      """Builds a CNN whose architecture and optimizer are controlled by 'hp'."""
      model = Sequential()
      model.add(Input(shape=X.shape[1:]))

      # Number of convolutional blocks: 2 to 4
      n_conv_layers = hp.Int("n_conv_layers", min_value=2, max_value=4, step=1)

      for i in range(n_conv_layers):
          filters = hp.Int(f"filters_{i}", min_value=32, max_value=256, step=32)
          kernel_size = hp.Choice(f"kernel_size_{i}", values=[3, 5])
          model.add(Conv2D(filters, (kernel_size, kernel_size), padding="same"))
          model.add(Activation("relu"))
          model.add(MaxPooling2D(pool_size=(2, 2)))

      model.add(Flatten())

      # Number of dense layers: 1 to 2, each with its own tunable width
      n_dense_layers = hp.Int("n_dense_layers", min_value=1, max_value=2, step=1)
      for j in range(n_dense_layers):
          dense_units = hp.Int(f"dense_units_{j}", min_value=32, max_value=256, step=32)
          model.add(Dense(dense_units))
          model.add(Activation("relu"))

      # Dropout to fight overfitting - the original model had none
      dropout_rate = hp.Float("dropout_rate", min_value=0.0, max_value=0.5, step=0.1)
      model.add(Dropout(dropout_rate))
```

```

# dropout to fight overfitting - the original model had none
dropout_rate = hp.Float("dropout_rate", min_value=0.0, max_value=0.5, steps=0.1)
model.add(Dropout(dropout_rate))

model.add(Dense(1))
model.add(Activation("sigmoid"))

learning_rate = hp.Float("learning_rate", min_value=1e-4, max_value=1e-2, sampling="log")
optimizer_choice = hp.Choice("optimizer", values=["adam", "rmsprop"])

if optimizer_choice == "adam":
    optimizer = tf.keras.optimizers.Adam(learning_rate=learning_rate)
else:
    optimizer = tf.keras.optimizers.RMSprop(learning_rate=learning_rate)

model.compile(loss="binary_crossentropy", optimizer=optimizer, metrics=["accuracy"])
return model

class CnnHyperModel(keras.Model):
    """Wraps build_model so batch size can be tuned alongside architecture."""

    def build(self, hp):
        return build_model(hp)

    def fit(self, hp, model, *args, **kwargs):
        batch_size = hp.Choice("batch_size", values=[32, 64, 128])
        return model.fit(*args, batch_size=batch_size, **kwargs)

```

```

[24]: tuner = kt.Hyperband(
    CnnHyperModel(),
    objectives="val_accuracy",
    max_epochs=15,
    factor=3,
    min_hypermodel_size=1,
    objectives="val_accuracy",
    max_epochs=15,
    factor=3,
    directory="kt_dir",
    project_name="dogs_vs_cats_tuning",
    overwrite=True,
)

tuner.search_space_summary()

stop_early = EarlyStopping(monitor="val_loss", patience=3, restore_best_weights=True)

tuner.search(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=15,
    callbacks=[stop_early],
    verbose=1,
)

Trial 30 Complete [00h 02m 14s]
val_accuracy: 0.7954000234603882

Best val_accuracy So Far: 0.868399977684821
Total elapsed time: 00h 58m 37s

```

```

[25]: best_hp = tuner.get_best_hyperparameters(num_trials=1)[0]

print("Best hyperparameters found:")
for param, value in best_hp.values.items():
    print(f" {param}: {value}")

Best hyperparameters found:
n_conv_layers: 3
filters_0: 96

```

```

Best hyperparameters found:
n_conv_layers: 3
filters_0: 96
kernel_size_0: 3
filters_1: 96
kernel_size_1: 5
n_dense_layers: 2
dense_units_0: 256
dropout_rate: 0.2
learning_rate: 0.000596921247059964
optimizer: adam
filters_2: 32
kernel_size_2: 5
dense_units_1: 128
batch_size: 64
filters_3: 256
kernel_size_3: 5
tuner/epochs: 35
tuner/initial_epoch: 0
tuner/bracket: 0
tuner/round: 0

```

Training the Tuned Model

We rebuild a fresh model with the winning hyperparameters and train it for longer, this time with two callbacks the original notebook didn't use:

- **EarlyStopping**: stops training once validation loss stops improving, restoring the best weights seen — this avoids wasting epochs (and overfitting) once the model has plateaued.
- **ReduceLRonPlateau**: halves the learning rate when validation loss stalls, which often squeezes out a bit more accuracy near the end of training.

```
[24]: best_model = tuner.hypermodel.build(best_hp)
      tuned_batch_size = best_hp.get("batch_size")
```

```
[26]: best_model = tuner.hypermodel.build(best_hp)
      tuned_batch_size = best_hp.get("batch_size")

      early_stop = EarlyStopping(monitor="val_loss", patience=5, restore_best_weights=True)
      reduce_lr = ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=2, min_lr=1e-6)

      tuned_history = best_model.fit(
          X_train, y_train,
          validation_data=(X_val, y_val),
          batch_size=tuned_batch_size,
          epochs=50,
          callbacks=[early_stop, reduce_lr],
      )
```

```

Epoch 1/50
313/313 — 23s 61ms/step - accuracy: 0.5990 - loss: 0.6553 - val_accuracy: 0.6766 - val_loss: 0.5949 - learning_rate: 5.9692e-04
Epoch 2/50
313/313 — 15s 49ms/step - accuracy: 0.7451 - loss: 0.5176 - val_accuracy: 0.7712 - val_loss: 0.4768 - learning_rate: 5.9692e-04
Epoch 3/50
313/313 — 16s 50ms/step - accuracy: 0.8160 - loss: 0.4068 - val_accuracy: 0.8136 - val_loss: 0.4036 - learning_rate: 5.9692e-04
Epoch 4/50
313/313 — 16s 50ms/step - accuracy: 0.8498 - loss: 0.3429 - val_accuracy: 0.8324 - val_loss: 0.3682 - learning_rate: 5.9692e-04
Epoch 5/50
313/313 — 16s 49ms/step - accuracy: 0.8750 - loss: 0.2911 - val_accuracy: 0.8446 - val_loss: 0.3468 - learning_rate: 5.9692e-04
Epoch 6/50
313/313 — 15s 48ms/step - accuracy: 0.9022 - loss: 0.2348 - val_accuracy: 0.8488 - val_loss: 0.3440 - learning_rate: 5.9692e-04
Epoch 7/50
313/313 — 16s 50ms/step - accuracy: 0.9247 - loss: 0.1849 - val_accuracy: 0.8482 - val_loss: 0.3803 - learning_rate: 5.9692e-04
Epoch 8/50
313/313 — 15s 48ms/step - accuracy: 0.9516 - loss: 0.1222 - val_accuracy: 0.8456 - val_loss: 0.4224 - learning_rate: 5.9692e-04
Epoch 9/50
313/313 — 16s 50ms/step - accuracy: 0.9829 - loss: 0.0511 - val_accuracy: 0.8486 - val_loss: 0.5310 - learning_rate: 2.9846e-04
Epoch 10/50
313/313 — 16s 50ms/step - accuracy: 0.9939 - loss: 0.0227 - val_accuracy: 0.8586 - val_loss: 0.6288 - learning_rate: 2.9846e-04
Epoch 11/50

```

```

Epoch 10/50 16s 50ms/step - accuracy: 0.9829 - loss: 0.0511 - val_accuracy: 0.8486 - val_loss: 0.5310 - learning_rate: 2.9846e-04
Epoch 11/50 16s 50ms/step - accuracy: 0.9939 - loss: 0.0227 - val_accuracy: 0.8586 - val_loss: 0.6288 - learning_rate: 2.9846e-04
Epoch 12/50 16s 51ms/step - accuracy: 0.9986 - loss: 0.0079 - val_accuracy: 0.8602 - val_loss: 0.6986 - learning_rate: 1.4923e-04

```

```

[27]: best_model.summary()

# Plot tuned model's training accuracy and loss
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(tuned_history.history['accuracy'], label='accuracy')
plt.plot(tuned_history.history['val_accuracy'], label='val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0, 1])
plt.legend(loc='lower right')
plt.title('Tuned Model: Training and Validation Accuracy')

plt.subplot(1, 2, 2)
plt.plot(tuned_history.history['loss'], label='loss')
plt.plot(tuned_history.history['val_loss'], label='val_loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(loc='upper right')
plt.title('Tuned Model: Training and Validation Loss')

plt.show()

# Evaluate on the held-out validation split (never used for training or search decisions
# beyond early stopping/LR scheduling, so this is a fair estimate of generalization)
val_pred_proba = best_model.predict(X_val).ravel()
val_pred = (val_pred_proba > 0.5).astype(int)

# Evaluate on the held-out validation split (never used for training or search decisions
# beyond early stopping/LR scheduling, so this is a fair estimate of generalization)
val_pred_proba = best_model.predict(X_val).ravel()
val_pred = (val_pred_proba > 0.5).astype(int)

cm_tuned = confusion_matrix(y_val, val_pred)
print("\nTuned Model Confusion Matrix (validation set):")
print(cm_tuned)

plt.figure(figsize=(5, 5))
plt.imshow(cm_tuned, interpolation='nearest', cmap=plt.cm.Blues)
plt.title('Tuned Model Confusion Matrix (Validation Set)')
plt.colorbar()
tick_marks = np.arange(len(CATEGORIES))
plt.xticks(tick_marks, CATEGORIES)
plt.yticks(tick_marks, CATEGORIES)
plt.ylabel('True label')
plt.xlabel('Predicted label')
thresh = cm_tuned.max() / 2.
for i, j in np.ndindex(cm_tuned.shape):
    plt.text(j, i, format(cm_tuned[i, j], 'd'),
             ha="center", va="center",
             color="white" if cm_tuned[i, j] > thresh else "black")
plt.tight_layout()
plt.show()

fpr_tuned, tpr_tuned, _ = roc_curve(y_val, val_pred_proba)
roc_auc_tuned = auc(fpr_tuned, tpr_tuned)

plt.figure(figsize=(6, 5))
plt.plot(fpr_tuned, tpr_tuned, color='darkorange', lw=2,
         label=f'ROC curve (area = {roc_auc_tuned:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])

```

```

plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Tuned Model: ROC Curve (Validation Set)')
plt.legend(loc='lower right')
plt.show()

```

Model: "sequential_1"

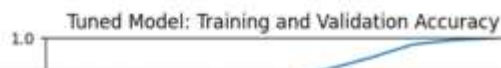
Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 100, 100, 96)	960
activation_5 (Activation)	(None, 100, 100, 96)	0
max_pooling2d_2 (MaxPooling2D)	(None, 50, 50, 96)	0
conv2d_3 (Conv2D)	(None, 50, 50, 96)	230,496
activation_6 (Activation)	(None, 50, 50, 96)	0
max_pooling2d_3 (MaxPooling2D)	(None, 25, 25, 96)	0
conv2d_4 (Conv2D)	(None, 25, 25, 32)	76,832
activation_7 (Activation)	(None, 25, 25, 32)	0
max_pooling2d_4 (MaxPooling2D)	(None, 12, 12, 32)	0
flatten_1 (Flatten)	(None, 4608)	0
max_pooling2d_4 (MaxPooling2D)	(None, 12, 12, 32)	0
flatten_1 (Flatten)	(None, 4608)	0
dense_3 (Dense)	(None, 256)	1,179,904
activation_8 (Activation)	(None, 256)	0
dense_4 (Dense)	(None, 128)	32,896
activation_9 (Activation)	(None, 128)	0
dropout_1 (Dropout)	(None, 128)	0
dense_5 (Dense)	(None, 1)	129
activation_10 (Activation)	(None, 1)	0

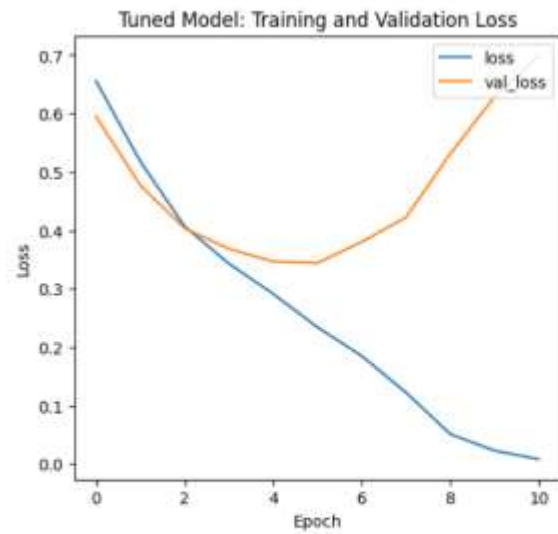
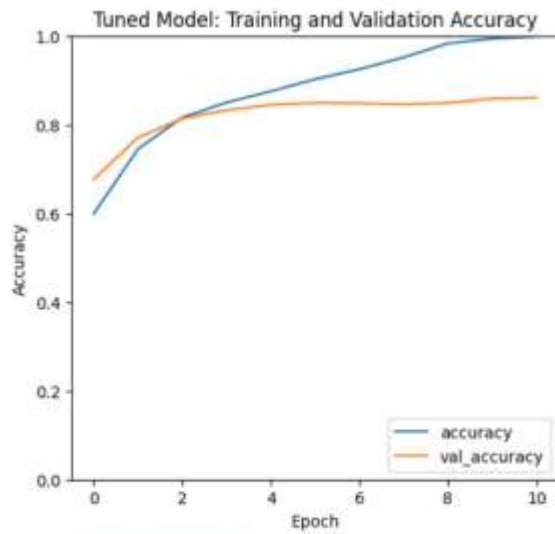
Total params: 4,563,653 (17.41 MB)

Trainable params: 1,521,217 (5.88 MB)

Non-trainable params: 0 (0.00 B)

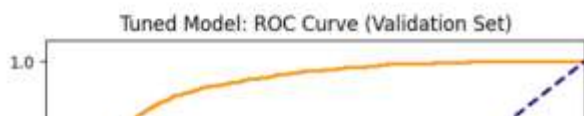
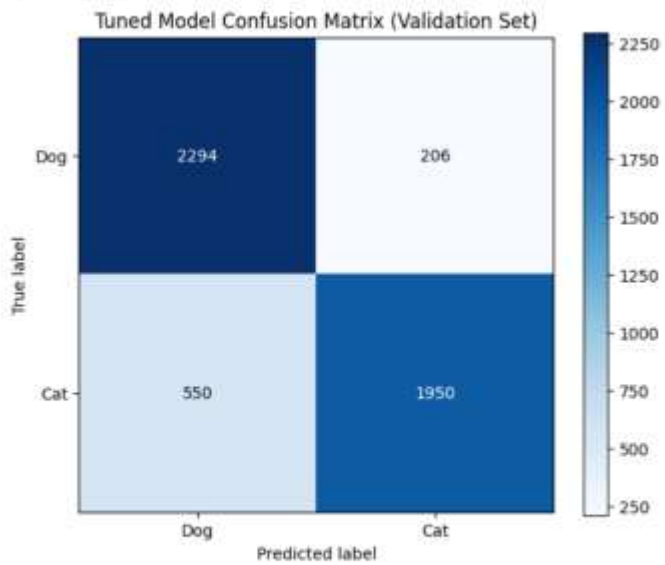
Optimizer params: 3,042,436 (11.61 MB)

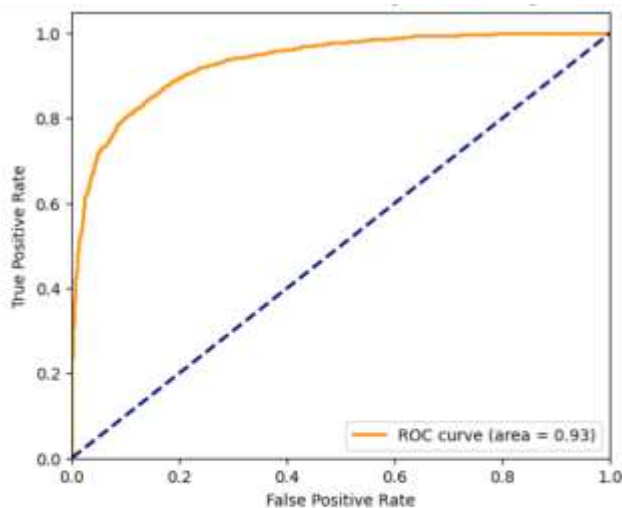




157/157 3s 8ms/step

Tuned Model Confusion Matrix (validation set):
[[2294 206]
 [550 1950]]





Baseline vs. Tuned: Quick Comparison

For an apples-to-apples comparison, the cell below re-evaluates the **original, untuned model** from the previous section on the same validation split (`X_val`, `y_val`) that Keras Tuner searched against, then compares it with the tuned model above. (The original training loop trained on the full dataset rather than `X_train`, so this is only an approximate comparison — but it's enough to see whether the search paid off.)

Keras Tuner searched against, then compares it with the tuned model above. (The original training loop trained on the full dataset rather than `X_train`, so this is only an approximate comparison — but it's enough to see whether the search paid off.)

```
[28]: baseline_val_pred_proba = model.predict(X_val).ravel()
baseline_val_pred = (baseline_val_pred_proba > 0.5).astype(int)
baseline_val_accuracy = (baseline_val_pred == y_val).mean()
baseline_fpr, baseline_tpr, _ = roc_curve(y_val, baseline_val_pred_proba)
baseline_auc = auc(baseline_fpr, baseline_tpr)

tuned_val_accuracy = (val_pred == y_val).mean()

print(f"Model: <22> | Val. Accuracy: >15 | Val. AUC: >12 |")
print(f"Baseline (manual): <22> | baseline_val_accuracy: >15.4% | baseline_auc: >12.4% |")
print(f"Tuned (Keras Tuner): <22> | tuned_val_accuracy: >15.4% | roc_auc_tuned: >12.4% |")

157/157 ————— 1a 4ms/step
Model          Val. Accuracy  Val. AUC
Baseline (manual) 0.8514      0.9296
Tuned (Keras Tuner) 0.8488      0.9335

[29]: tuned_model_filename = 'dog_cat_classifier_tuned.keras'
best_model.save(tuned_model_filename)
print(f"Tuned model saved to {tuned_model_filename}")

Tuned model saved to dog_cat_classifier_tuned.keras
```

6. Overall Justification / Remarks

The notebook uses a 100×100 grayscale input representation and the original baseline CNN uses three convolutional blocks with 2, 64, and 64 filters, 3×3 kernels, 2×2 max pooling, two 64-unit dense layers, sigmoid output, binary cross-entropy, Adam, batch size 128, and 5 epochs. The hyperparameter-tuning section then uses Keras Tuner Hyperband to search the number of convolutional blocks, filters, kernel sizes, dense layers/units, dropout, learning rate, optimizer, and batch size. The best configuration reported by the tuner has 3 convolutional blocks (96 filters/3×3, 96 filters/5×5, 32 filters/5×5), two dense layers (256 and 128 units), dropout 0.2, Adam, learning rate 0.000596921247059964, and batch size 64. The tuner's best validation accuracy reached 0.8684 during search. The final tuned model was retrained with EarlyStopping (patience 5, restore_best_weights=True) and ReduceLROnPlateau (factor 0.5, patience 2, minimum learning rate 1e-6). The final evaluation reports 0.8488 validation accuracy and 0.9335 AUC, with a validation confusion matrix of [[2294, 206], [550, 1950]]. The training log shows a growing gap between training and validation performance: by epoch 11, training accuracy reached 0.9986 while validation accuracy was 0.8602 and validation loss had increased to 0.6986. This pattern indicates overfitting pressure, which is why restoring the best validation-loss weights is important..